

Threat Modeling Report

Created on 10-Mar-26 5:24:58 PM

Threat Model Name:
Owner:
Reviewer:
Contributors:
Description:
Assumptions:
External Dependencies:

Threat Model Summary:

Not Started	0
Not Applicable	0
Needs Investigation	28
Mitigation Implemented	0
Total	28
Total Migrated	0

Diagram: Diagram 1

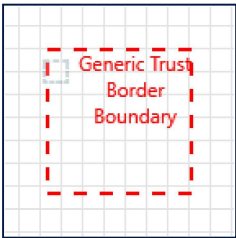


Diagram 1 Diagram Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	0
Total Migrated	0

Diagram: Diagram 2

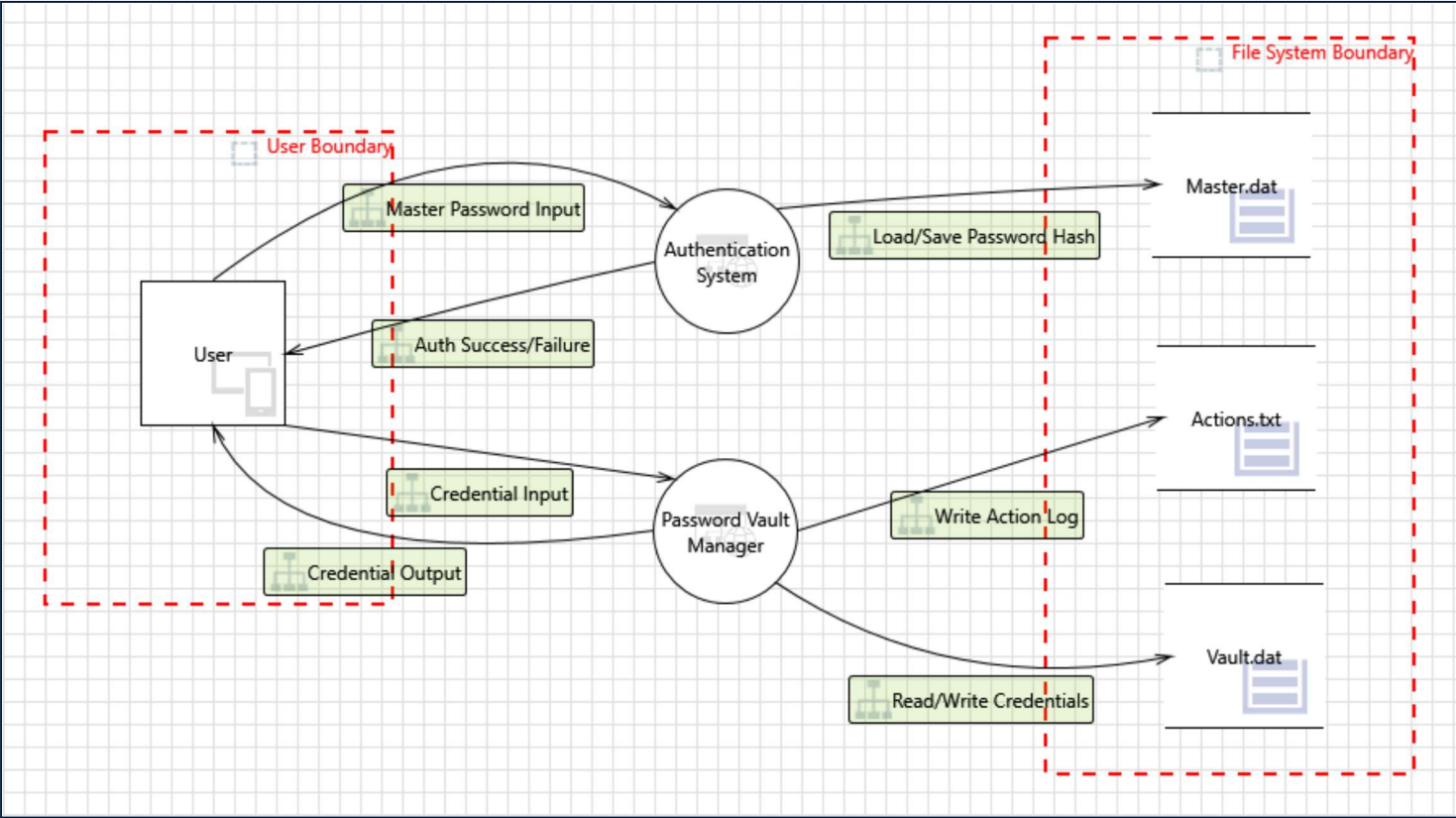
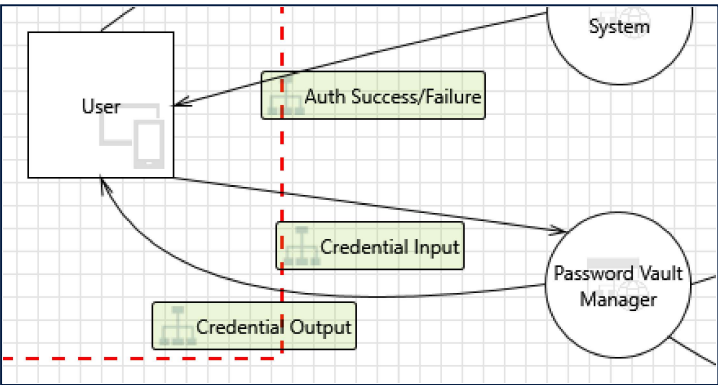


Diagram 2 Diagram Summary:

Not Started	0
Not Applicable	0
Needs Investigation	28
Mitigation Implemented	0
Total	28
Total Migrated	0

Interaction: Credential Input



1. An adversary may jail break into a mobile device and gain elevated privileges [State: Needs Investigation] [Priority: High]

Category: Elevation of Privileges
Description: An adversary may jail break into a mobile device and gain elevated privileges
Justification: An adversary could exploit improper input validation on credential fields to inject malicious data and gain elevated access to the Password Vault Manager.

Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: <https://aka.ms/tmtauthz#rooting-detection>

SDL Phase: Design

2. An adversary can reverse weakly encrypted or hashed content [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: An adversary could intercept credential input data containing site names, usernames and passwords if the data flow between the user and the vault manager is not protected.

Possible Mitigation(s): Do not expose security details in error messages. Refer: <https://aka.ms/tmtxmgmt#messages> Implement Default error handling page. Refer: <https://aka.ms/tmtxmgmt#default> Set Deployment Method to Retail in IIS. Refer: <https://aka.ms/tmtxmgmt#deployment> Use only approved symmetric block ciphers and key lengths. Refer: <https://aka.ms/tmtcrypto#cipher-length> Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: <https://aka.ms/tmtcrypto#vector-ciphers> Use approved asymmetric algorithms, key lengths, and padding. Refer: <https://aka.ms/tmtcrypto#padding> Use approved random number generators. Refer: <https://aka.ms/tmtcrypto#numgen> Do not use symmetric stream ciphers. Refer: <https://aka.ms/tmtcrypto#stream-ciphers> Use approved MAC/HMAC/keyed hash algorithms. Refer: <https://aka.ms/tmtcrypto#mac-hash> Use only approved cryptographic hash functions. Refer: <https://aka.ms/tmtcrypto#hash-functions> Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

3. An adversary may gain access to sensitive data from log files [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: An adversary may gain access to credential data in transit if the application does not enforce secure memory handling and input protection mechanisms.

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: <https://aka.ms/tmtauditlog#log-sensitive-data> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access>

SDL Phase: Implementation

4. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client

Justification: An adversary can gain access to sensitive credential data if certificate pinning is not implemented, allowing interception of data flowing into the vault manager.

Possible Mitigation(s): Implement Certificate Pinning. Refer: <https://aka.ms/tmtcommsec#cert-pinning>

SDL Phase: Implementation

5. An adversary can gain sensitive data from mobile device [State: Needs Investigation] [Priority: High]**Category:** Information Disclosure**Description:** If application saves sensitive PII or HBI data on phone SD card or local storage, then it ay get stolen.**Justification:** If the application does not encrypt sensitive credential data in memory, an adversary could extract plaintext usernames and passwords from the process memory at runtime**Possible Mitigation(s):** Encrypt sensitive or PII data written to phones local storage. Refer: https://aka.ms/tmtdata#pii-phones**SDL Phase:** Implementation**6. An adversary can gain access to sensitive information through error messages [State: Needs Investigation] [Priority: High]****Category:** Information Disclosure**Description:** An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details**Justification:** An adversary could gain access to security-sensitive credential data through verbose error messages or unhandled exceptions if proper error handling is not implemented.**Possible Mitigation(s):** Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum**SDL Phase:** Implementation**7. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Needs Investigation] [Priority: Medium]****Category:** Repudiation**Description:** Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system**Justification:** Without comprehensive audit logging on credential operations, an attacker could deny having added, modified or deleted vault entries leaving no traceable evidence of tampering.**Possible Mitigation(s):** Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management**SDL Phase:** Implementation**8. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Needs Investigation] [Priority: High]****Category:** Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: An adversary could spoof the Password Vault Manager process to intercept credential input from the user if proper process authentication is not enforced.

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

9. An adversary can steal sensitive data like user credentials [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: An attacker could steal authentication tokens or session identifiers to impersonate a legitimate user and submit fraudulent credential entries into the vault

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: <https://aka.ms/tmtdata#autocomplete-input> Perform input validation and filtering on all string type Model properties. Refer: <https://aka.ms/tmtinputval#typemodel> Validate all redirects within the application are closed or done safely. Refer: <https://aka.ms/tmtinputval#redirect-safe> Enable step up or adaptive authentication. Refer: <https://aka.ms/tmtauthn#step-up-adaptive-authn> Implement forgot password functionalities securely. Refer: <https://aka.ms/tmtauthn#forgot-pword-fxn> Ensure that password and account policy are implemented. Refer: <https://aka.ms/tmtauthn#pword-account-policy> Implement input validation on all string type parameters accepted by Controller methods. Refer: <https://aka.ms/tmtinputval#string-method>

SDL Phase: Implementation

10. An adversary can create a fake website and launch phishing attacks [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: A phishing attack could present a fake vault interface to the user, tricking them into entering credentials that are captured by the attacker instead of stored securely.

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts> Ensure that authenticated ASPNET pages incorporate UI Redressing or clickjacking defences. Refer: <https://aka.ms/tmtconfigmgmt#ui-defenses> Validate all redirects within the application are closed or done safely. Refer: <https://aka.ms/tmtinputval#redirect-safe>

SDL Phase: Implementation

11. An adversary may spoof User and gain access to Web Application [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: If proper authentication is not enforced on the credential input channel, an adversary could spoof a legitimate user identity and inject unauthorized entries into the vault.

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

12. An adversary can reverse engineer and tamper binaries [State: Needs Investigation] [Priority: High]

Category: Tampering

Description: An adversary can use various tools, reverse engineer binaries and abuse them by tampering

Justification: An adversary could reverse engineer the vault binary to bypass input validation and tamper with credential data before it is encrypted and stored in Vault.dat.

Possible Mitigation(s): Obfuscate generated binaries before distributing to end users. Refer: https://aka.ms/tmtdata#binaries-end

SDL Phase: Design

13. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Needs Investigation] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: SQL injection style input manipulation could be used to tamper with credential fields if type-safe parameter handling and input sanitization are not enforced in the vault manager.

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

14. An adversary can gain access to sensitive data stored in Web App's config files [State: Needs Investigation] [Priority: High]

Category: Tampering

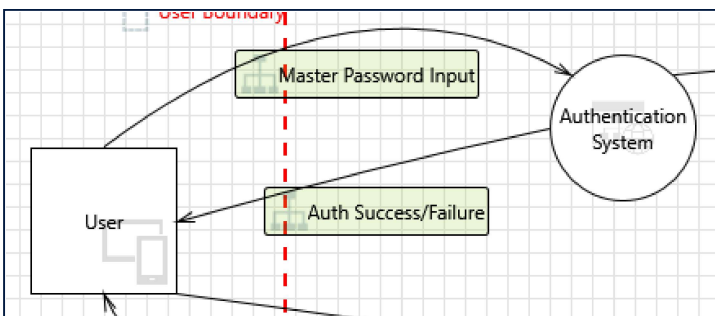
Description: An adversary can gain access to the config files, and if sensitive data is stored in it, it would be compromised.

Justification: An adversary could tamper with credential input by intercepting and modifying data before it reaches the Password Vault Manager if the input channel is not properly secured.

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

Interaction: Master Password Input



15. An adversary may jail break into a mobile device and gain elevated privileges [State: Needs Investigation] [Priority: High]

Category: Elevation of Privileges

Description: An adversary may jail break into a mobile device and gain elevated privileges

Justification: An adversary could exploit the weak master password hashing (simple multiply-by-31 hash) to gain elevated access to the vault and all stored credentials.

Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: https://aka.ms/tmtauthz#rooting-detection

SDL Phase: Design

16. An adversary can reverse weakly encrypted or hashed content [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: An adversary could reverse the weak custom hash stored in Master.dat since it uses a simple algorithm, exposing the master password.

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslTls

SDL Phase: Implementation

17. An adversary may gain access to sensitive data from log files [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: An adversary may gain access to the master password transmitted via console input if the system is being monitored or logged by a third party.

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

18. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client

Justification: An adversary can gain access to sensitive authentication data if certificate pinning or secure communication is not implemented between the user and the authentication system.

Possible Mitigation(s): Implement Certificate Pinning. Refer: <https://aka.ms/tmtcommsec#cert-pinning>

SDL Phase: Implementation

19. An adversary can gain sensitive data from mobile device [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: If application saves sensitive PII or HBI data on phone SD card or local storage, then it ay get stolen.

Justification: If the application does not properly protect sensitive data in memory, an adversary could extract the master password from process memory during runtime.

Possible Mitigation(s): Encrypt sensitive or PII data written to phones local storage. Refer: <https://aka.ms/tmtdata#pii-phones>

SDL Phase: Implementation

20. An adversary can gain access to sensitive information through error messages [State: Needs Investigation] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: An adversary could gain access to security-sensitive data exposed through error messages or debug output if proper input validation and error handling are not enforced.

Possible Mitigation(s): Do not expose security details in error messages. Refer: <https://aka.ms/tmtxmgmt#messages> Implement Default error handling page. Refer: <https://aka.ms/tmtxmgmt#default> Set Deployment Method to Retail in IIS. Refer: <https://aka.ms/tmtxmgmt#deployment> Exceptions should fail safely. Refer: <https://aka.ms/tmtxmgmt#fail> ASP.NET applications must disable tracing and debugging prior to deployment. Refer: <https://aka.ms/tmtconfigmgmt#trace-deploy> Implement controls to prevent username enumeration. Refer: <https://aka.ms/tmtauthn#controls-username-enum>

SDL Phase: Implementation

21. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Needs Investigation] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: Without proper audit logging on authentication attempts, an attacker could deny unauthorized access attempts to the vault leaving no traceable evidence.

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: <https://aka.ms/tmtauditlog#auditing> Ensure that log rotation and separation are in place. Refer: <https://aka.ms/tmtauditlog#log-rotation> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access> Ensure that User Management Events are Logged. Refer: <https://aka.ms/tmtauditlog#user-management>

SDL Phase: Implementation

22. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: An adversary could spoof the authentication system by presenting forged credentials, bypassing identity verification if X.509 certificates or strong authentication mechanisms are absent.

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

23. An adversary can steal sensitive data like user credentials [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: An attacker could steal valid session or authentication tokens to impersonate a legitimate user and gain unauthorized access to the password vault.

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

24. An adversary can create a fake website and launch phishing attacks [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: Phishing attacks could trick the user into entering their master password into a fake application interface, compromising vault access credentials.

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

25. An adversary may spoof User and gain access to Web Application [State: Needs Investigation] [Priority: High]

Category:	Spoofing
Description:	If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application
Justification:	If proper authentication is not enforced, an adversary may spoof the identity of a legitimate user and gain unauthorized access to the authentication system.
Possible Mitigation(s):	Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app
SDL Phase:	Design

26. An adversary can reverse engineer and tamper binaries [State: Needs Investigation] [Priority: High]

Category:	Tampering
Description:	An adversary can use various tools, reverse engineer binaries and abuse them by tampering
Justification:	An adversary could reverse engineer or tamper with the authentication binary to bypass the master password check and gain direct access to the vault.
Possible Mitigation(s):	Obfuscate generated binaries before distributing to end users. Refer: https://aka.ms/tmtdata#binaries-end
SDL Phase:	Design

27. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Needs Investigation] [Priority: High]

Category:	Tampering
Description:	SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.
Justification:	SQL injection or input manipulation could be used to tamper with the authentication input if type-safe parameter handling is not enforced in the system.
Possible Mitigation(s):	Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe
SDL Phase:	Implementation

28. An adversary can gain access to sensitive data stored in Web App's config files [State: Needs Investigation] [Priority: High]

Category:	Tampering
Description:	An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.
Justification:	An adversary could tamper with the master password input channel to intercept or modify credentials before they reach the authentication system if encryption is not applied.
Possible Mitigation(s):	Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data
SDL Phase:	Implementation